

InfraDrone Engine Pipeline

Luke Pitstick

June 13, 2026

1 Overview

InfraDrone is an image-processing and machine-learning pipeline for finding road damage from drone or camera imagery. The engine starts with a raw frame, cleans it up for model inference, detects candidate damage regions, segments the exact crack or pothole shape, and then converts those model outputs into structured damage records.

The important part is that the engine does not stop at “there is a crack somewhere in this box.” The detection model gives the rough location, but the segmentation and post-processing stages are what make the result useful. Those stages estimate the crack geometry, split and merge branches, classify the crack subtype, and calculate measurements like length, thickness, and area.

2 Pipeline

At a high level, the pipeline is:

Step	Purpose
1	Load a frame from the input queue or image source.
2	Preprocess the image for road-surface contrast and model input size.
3	Run YOLO detection to find cracks and potholes as bounding boxes.
4	Crop each detected region so segmentation only sees the relevant area.
5	Run YOLO segmentation to create binary masks for each damage region.
6	Skeletonize the masks and analyze crack branches.
7	Merge branch fragments that are close and directionally aligned.
8	Measure dimensions and classify the damage subtype.
9	Return a structured Damage record for downstream reporting.

In code, the top-level **Engine** wires together **DetectionEngine** and **SegmentationEngine**. The current implementation is:

$$I \rightarrow I' \rightarrow D \rightarrow \{C_i\} \rightarrow \{M_i\} \rightarrow \{S_i\} \rightarrow \{B_i\} \rightarrow \text{Damage}$$

where I is the original image, I' is the preprocessed image, D is the detection output, C_i are cropped damage regions, M_i are segmentation masks, S_i are skeletonized masks, and B_i are final crack branches.

3 Preprocessing

The preprocessing step is meant to make road texture easier for the models to read. A raw frame can have glare, shadows, compression noise, and inconsistent resolution, so each image is normalized before inference.

The current preprocessing function does three things:

- resizes the image to the model input size;
- applies CLAHE in LAB color space to improve local contrast;
- applies bilateral filtering to denoise while preserving edges.

Mathematically, I think of it as:

$$I' = \text{Denoise}(\text{CLAHE}(\text{Resize}(I))).$$

This is a practical step more than a fancy one. Cracks are thin, low-contrast structures, so increasing local contrast before detection and segmentation helps the model see the features I care about.

4 Detection

The detection stage uses a YOLO model trained to find two high-level classes:

Class ID	Label
0	Crack
1	Pothole

For a preprocessed image I' , the detector returns bounding boxes:

$$D = \{(x_1, y_1, x_2, y_2, c, t)\},$$

where c is confidence and t is the predicted damage type. Detections below the configured confidence threshold are removed. The remaining boxes are cropped from the image and sent into segmentation.

This two-model design keeps the segmentation problem smaller. Instead of asking the segmentation model to understand the whole drone image, the detector narrows the input down to likely damage regions first.



Figure 1: Detection output for a road crack. The blue box is the detector’s crop proposal, and the class label plus confidence score are the metadata carried into the next stage.

5 Segmentation

The segmentation model returns a binary mask for each detected damage crop. Each mask is thresholded and skeletonized:

$$M = \mathbf{1}(P > 0.5), \quad S = \text{Skeletonize}(M).$$

The mask M keeps the full damaged area, while the skeleton S reduces the crack to a one-pixel-wide centerline. That centerline is much easier to reason about because crack length, direction, endpoints, and branches become graph-like properties instead of blob properties.

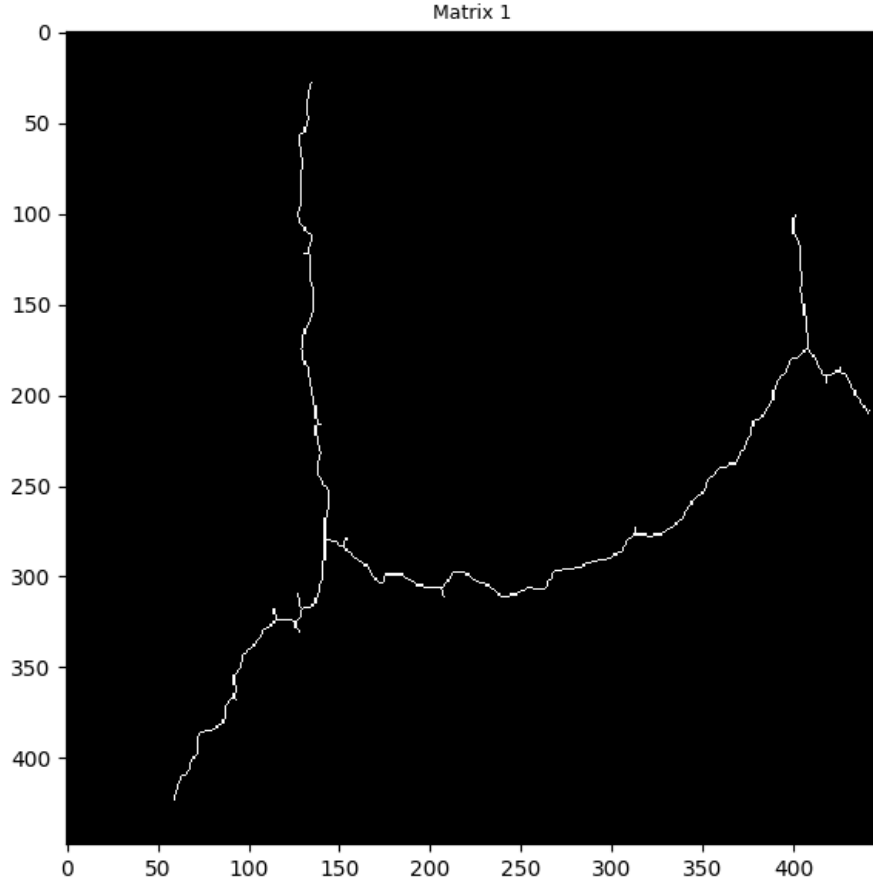


Figure 2: Skeletonized crack mask. The full segmentation mask is reduced to a one-pixel-wide centerline while preserving the shape of the crack.

6 Branch Detection

Once the crack is skeletonized, the engine searches for branch points and endpoints. A branch point is a skeleton pixel with at least three neighboring skeleton pixels in its 8-connected neighborhood:

$$N(p) = \sum_{q \in \mathcal{N}_8(p)} S(q), \quad p \text{ is a branch point if } S(p) = 1 \text{ and } N(p) \geq 3.$$

The engine dilates the branch-point region, removes it from the skeleton, and then labels the remaining connected components. Each component becomes a candidate branch. Very short components are removed so tiny artifacts do not become reported cracks.

Branch Count: 11

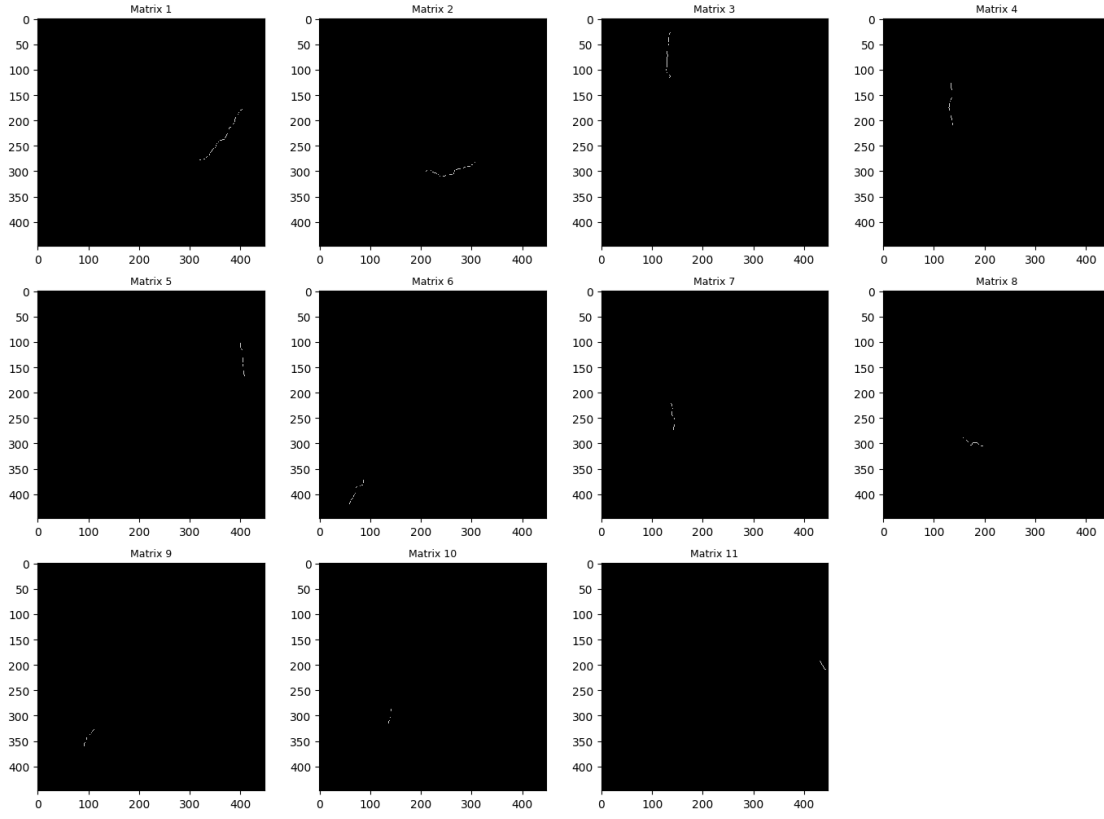


Figure 3: Skeleton split into branch segments. Junction pixels are removed so connected-component labeling can isolate each branch.

Branch Count: 24

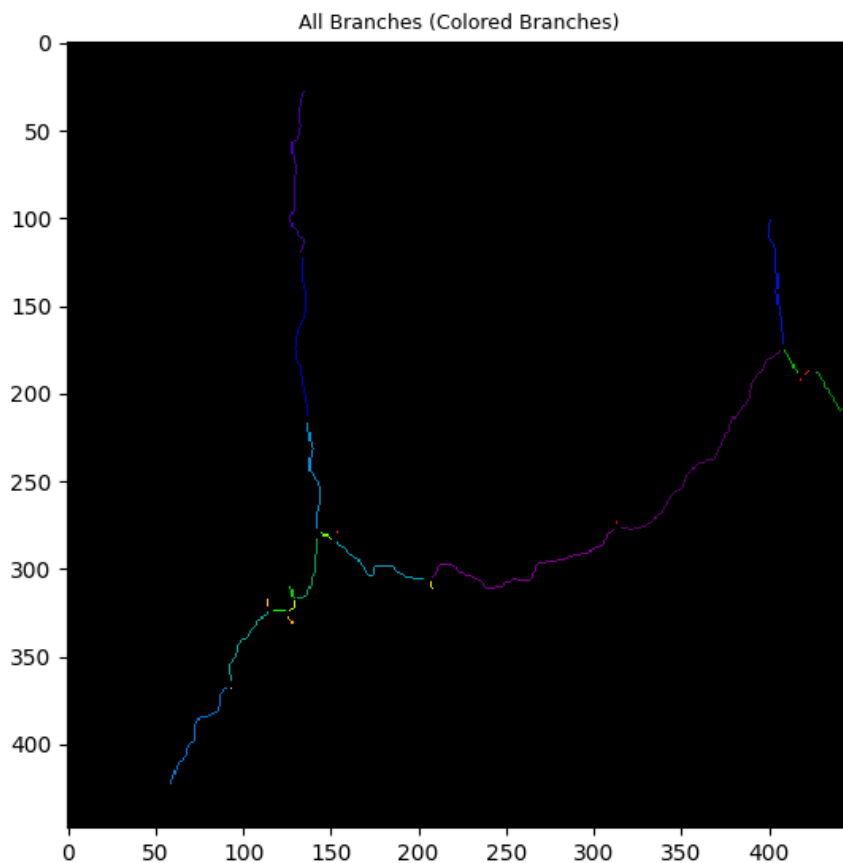


Figure 4: Colored branch labels after pruning. Each surviving component is treated as a separate branch candidate.

7 Branch Angles and Merging

Cracks often get split into multiple pieces because segmentation masks are imperfect. The engine tries to reconnect branch fragments when they are close together and point in roughly the same direction.

For each branch, the overall axis angle is estimated from the farthest pair of endpoints:

$$\theta = \text{atan2}(r_2 - r_1, c_2 - c_1) \bmod 180^\circ.$$

The angle is treated as undirected, so 0° and 180° are equivalent. For local merge checks, the engine also estimates endpoint direction with PCA over nearby skeleton pixels. This gives a better tangent estimate at the endpoint than using the whole branch.

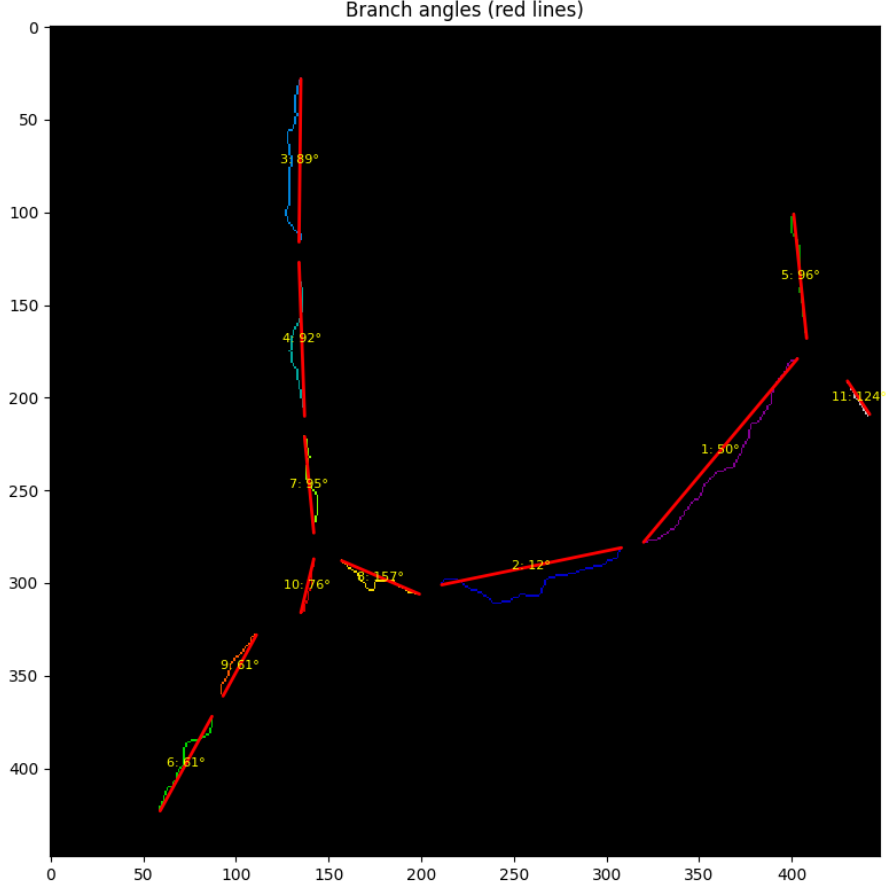


Figure 5: Estimated branch axes. The red guide lines show the direction used for classification and merge checks.

Two branches can merge if their closest endpoint distance and local angle difference are below the configured thresholds:

$$d(e_i, e_j) < \tau_d, \quad \Delta\theta(\theta_i, \theta_j) < \tau_\theta.$$

When more than one pair is eligible, the engine chooses the lowest score:

$$\text{score} = d(e_i, e_j) + \Delta\theta(\theta_i, \theta_j).$$

The selected pair is merged by drawing a bridge between the closest endpoints, recomputing the skeleton, and repeating until no more pairs qualify.

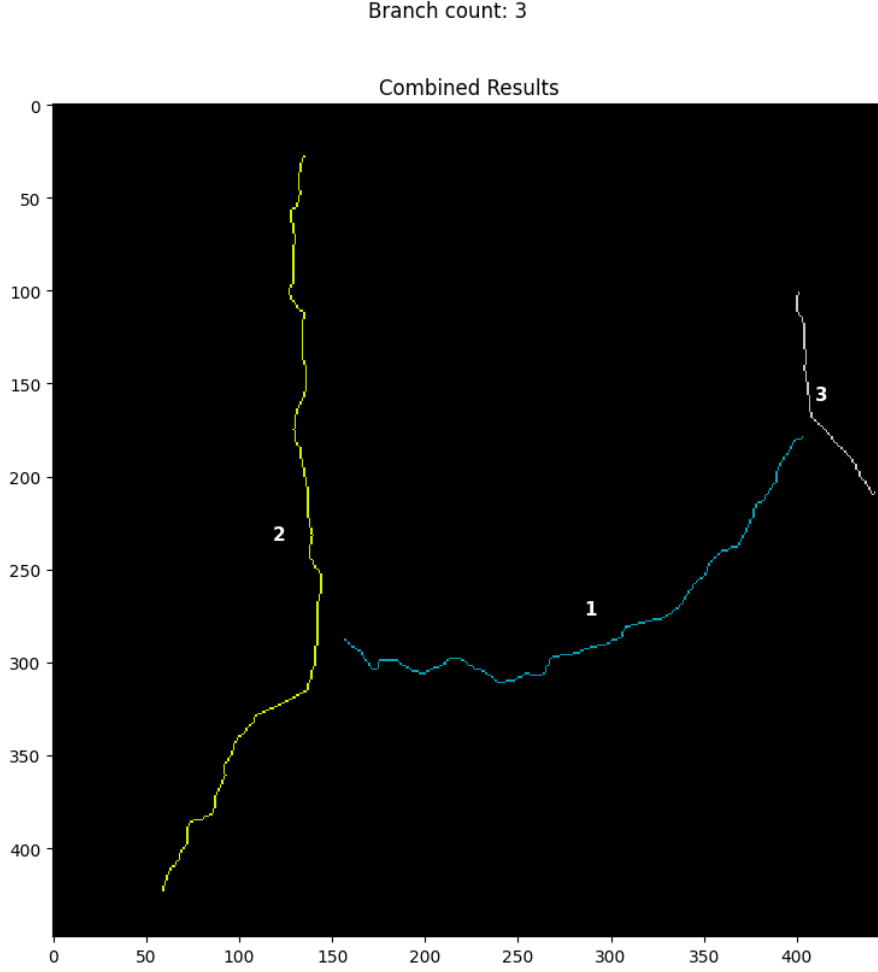


Figure 6: Final branch groups after merging. The numbered branches are the geometry that gets passed into measurement and subtype classification.

8 Measurements

The final branch mask and skeleton are converted into physical measurements. Right now the engine stores these in pixel units unless a real-world scale ratio is configured.

8.1 Thickness

Thickness is estimated with a distance transform. For each skeleton pixel, the distance transform gives the distance to the nearest mask boundary. Doubling that value gives a local width estimate:

$$w = \frac{1}{|S|} \sum_{p \in S} 2 \operatorname{dist}(p, \partial M).$$

This gives an average crack width across the skeleton.

8.2 Length

Length is currently estimated as the Euclidean distance between the farthest endpoint pair:

$$L = \max_{e_i, e_j \in E} \|e_i - e_j\|_2.$$

This works well for mostly straight branches. A future improvement would be to measure length along the skeleton path itself, especially for curved cracks.

8.3 Area

Area is the number of foreground pixels in the mask:

$$A = \sum_p M(p).$$

If the engine has a pixel-to-real-world conversion ratio, thickness, length, and area can be scaled into centimeters or inches.

9 Subtype Classification

After the geometry is measured, the engine assigns a crack subtype. The current rule is simple:

- if the branch network has many connections, classify it as alligator cracking;
- otherwise classify by orientation angle;
- branches above the angle threshold are longitudinal;
- branches below the angle threshold are transverse.

This is intentionally explainable. It is not trying to hide the decision in another model. The crack subtype comes from the skeleton shape and its orientation in the image.

10 Output Record

Each final result becomes a **Damage** object. The record includes:

- a unique ID;

- the final mask and skeleton;
- the damage type, either crack or pothole;
- confidence;
- thickness, length, and area;
- subtype;
- stress range;
- number of branch connections;
- severity placeholder.

The severity field is currently set to zero. The planned direction is to combine measured damage geometry with road-class stress information and fatigue-growth modeling.

11 Severity Direction

The project is set up for a later severity layer based on fatigue crack growth. The intended model is Paris' Law:

$$\frac{da}{dN} = C(\Delta K)^m.$$

Here, a is crack length, N is load cycles, C and m are material constants, and ΔK is the stress-intensity range. In the engine, the **StressRange** enum gives a first pass at road class, from residential roads up through freeways.

The idea is that the same observed crack should not always mean the same priority. A crack on a low-traffic residential road and a similar crack on a freeway should be treated differently because the expected stress cycles and consequences are different.

12 Current Limitations

The current pipeline is working as an end-to-end analysis path, but there are still a few areas that need to be tightened:

- severity scoring is still a placeholder;
- the backend persistence method is stubbed out;
- the frame listener is in transition from a database-style queue to Redis;
- length is measured endpoint-to-endpoint instead of along the skeleton path;
- the real-world pixel scale needs to come from camera altitude, calibration, or metadata before centimeter-level dimensions are meaningful.

13 Conclusion

InfraDrone’s pipeline is built around a simple split: use detection to find where damage probably is, then use segmentation and image processing to understand what that damage actually looks like. The screenshots show why the post-processing matters. The raw mask is useful, but the skeleton, branches, angles, and merged crack paths are what turn a model prediction into something that can be measured and reported.